

INTRODUCTION TO DEVOPS

OBJECTIVES

By the end of this 1-day session, participants will:

- Understand the **core concepts** and goals of DevOps
- Learn about the **CI/CD pipeline** and how automation plays a key role
- Explore tools like **Jenkins, Docker, Selenium, and Terraform**
- Understand the importance of **testing, monitoring, and infrastructure automation**
- Get introduced to **microservices architecture** and how DevOps supports it
- See how these practices come together to deliver software **faster and more reliably**

AGENDA

- DevOps Overview
- CI/CD with Jenkins
- Containerization with Docker
- Automated Testing
- Infrastructure as Code (IaC)
- Monitoring in DevOps
- Microservices Architecture

THE WORLD BEFORE DEVOPS

- Before DevOps became a thing, software development followed what was known as the waterfall methodology. Essentially, it means that each development phase was handled by a different team or department.
- In those days, software releases were often a stressful and unpredictable process. Developers would throw their code over the wall to operations and hope for the best, while operations teams would struggle to deploy it smoothly without context or understanding of the code.
- As you can imagine, little dev and ops cooperation led to many delays and miscommunication, with projects often running over budget and behind schedule.

THE EARLY DAYS OF SOFTWARE DEVELOPMENT

- Requirements gathering and analysis: This phase involves gathering and documenting.
- Design: In this phase, the system architecture and design are developed.
- Implementation: This phase involves the actual coding and development of the system or software.
- Testing: The system is tested in this phase.
- Deployment: The system is deployed and available to the end-users in this phase.
- Maintenance: This phase involves ongoing maintenance, bug fixes, and updates to the system.

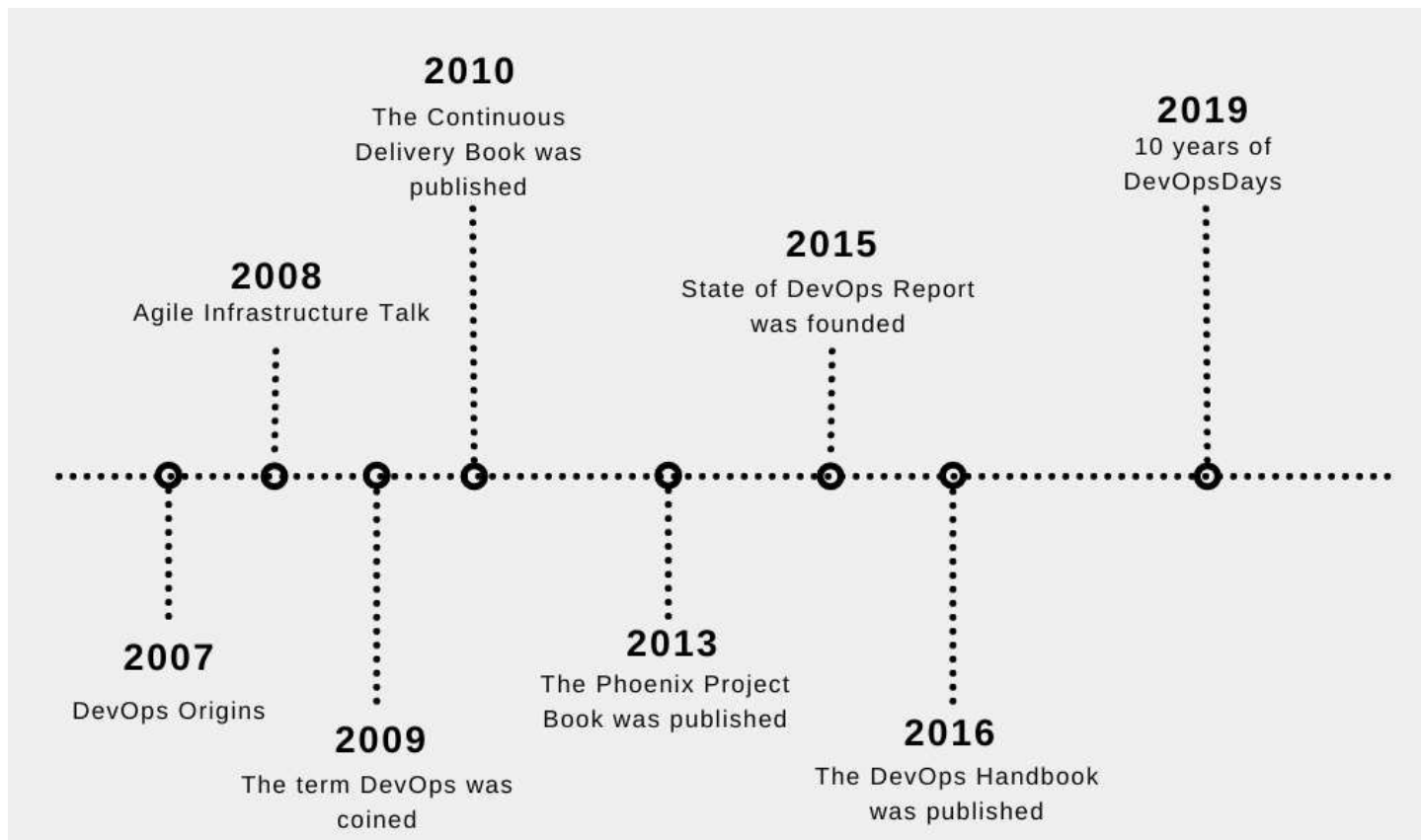


THE RISE OF THE AGILE METHODOLOGY

- Agile combats the rigidity of the Waterfall model by focusing on the clean delivery of individual pieces or parts of the software and not the entire application.
- Teams were encouraged to work in short, iterative cycles, known as “Sprints”, and to continuously refine and improve the software they were building.



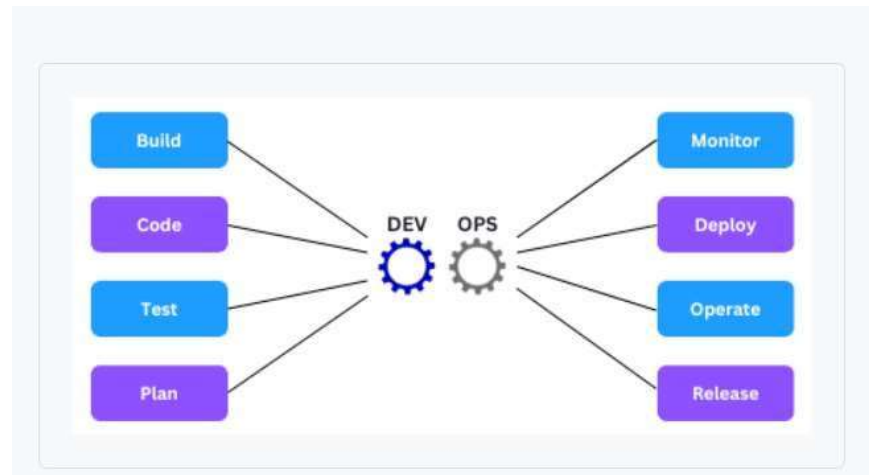
HISTORY OF DEVOPS



INTRODUCTION TO DEVOPS AND BENEFITS

What is DevOps?

- DevOps is a practice that enables a single team to handle the whole application lifecycle, including development, testing, release, deployment, operation, display, and planning.
- It is a mix of the terms “Dev” (for development) and “Ops” (for operations). We can speed up the delivery of applications and services by a business with the aid of DevOps.
- Amazon, Netflix, and other businesses have all effectively embraced DevOps to improve their customer experience.



DEVOPS CAMS FRAMEWORK

DevOps Culture

DevOps Automation

DevOps Measurement

DevOps Sharing

DEVOPS CULTURE

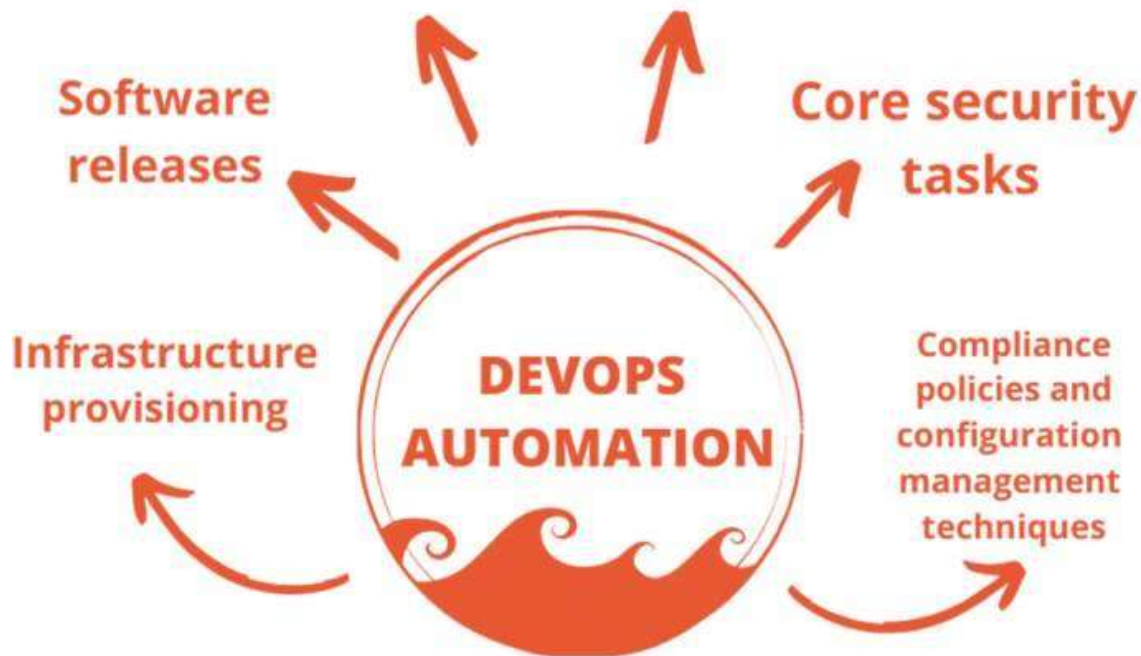


Key points :

- Shared Ownership
- Focus is on collaboration
- Open Communication
- Business Alignment
- Continuous Learning
- Team Autonomy
- Empathy and Trust
- Faster and Reliable Delivery

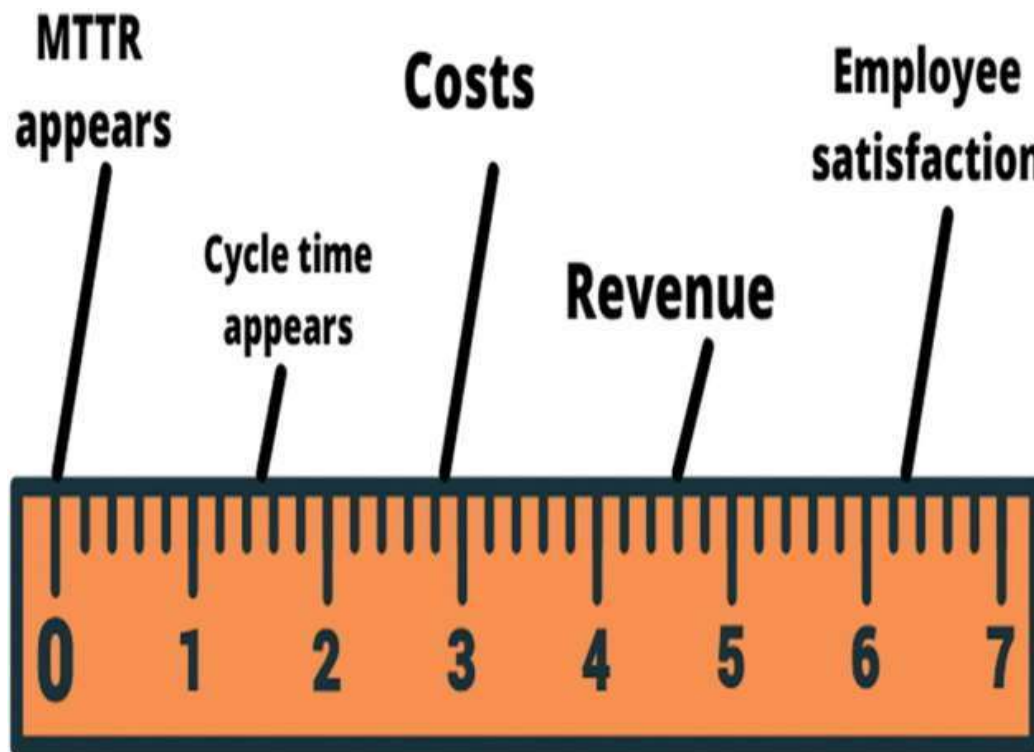
DEVOPS AUTOMATION

Testing Deployment



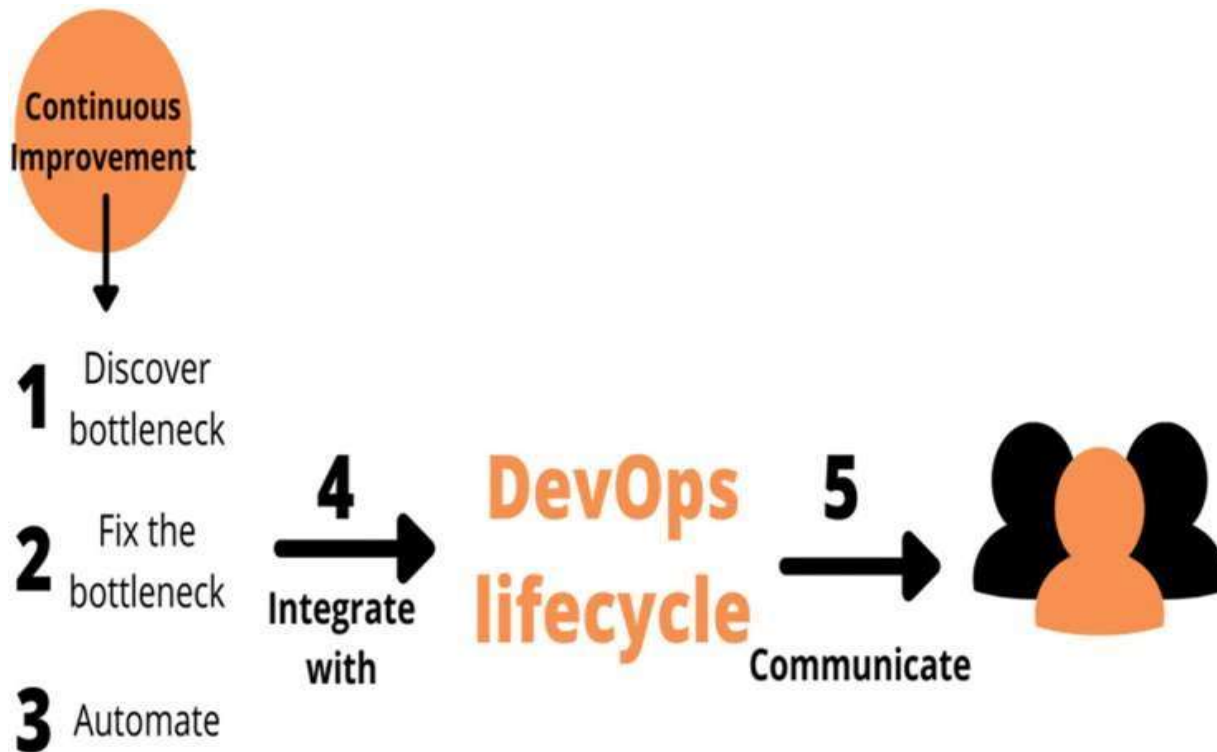
Replacing manual work
Faster and fewer errors
Freeing up people

DEVOPS MEASUREMENT



- It's the core of DevOps improvement.
- Involves continuous monitoring and analysis.
- Helps identify faults, gather feedback, and support team alignment.
- Metrics include MTTR, lead time, costs, and satisfaction.
- Enables real improvement, not just assumptions.

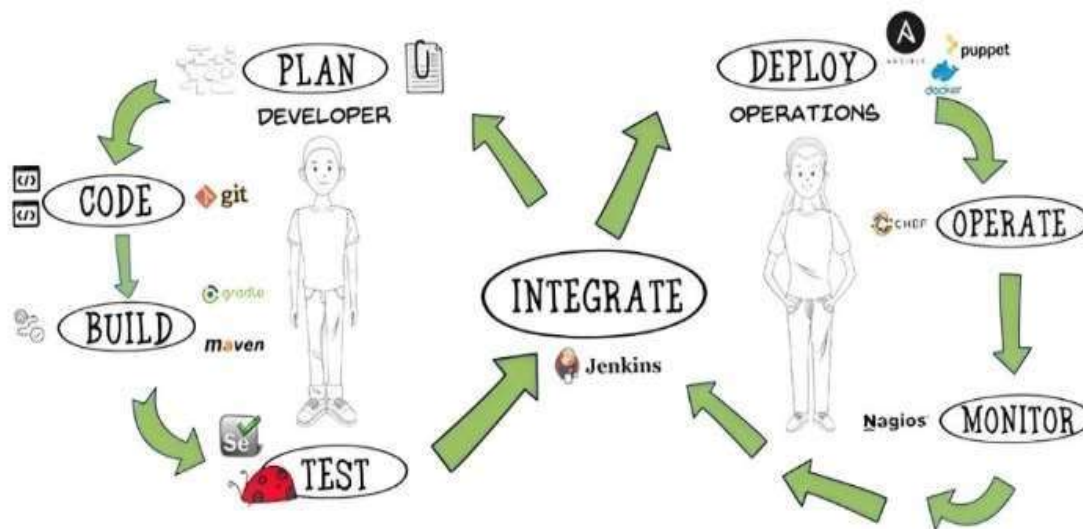
DEVOPS SHARING



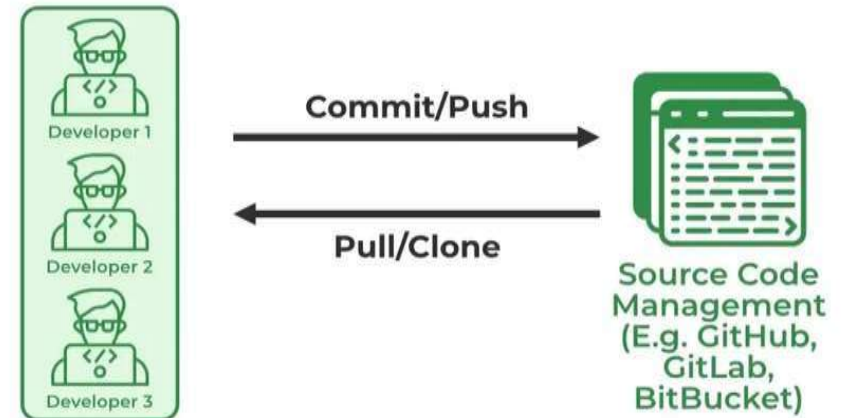
- Breaking down information silos
- Knowledge transfer
- Continuous feedback
- Transparency

DEVOPS PHASES

The following are the steps at how the DevOps lifecycle functions at each phase



7 Cs OF DEVOPS



Continuous development

CONTINUOUS DEPLOYMENT vs. CONTINUOUS DELIVERY

- **Continuous Deployment:** It is the process of automatically deploying an application into the production environment when it has completed testing and the build stages. Here, we'll automate everything from obtaining the application's source code to deploying it.
- **Continuous Delivery:** It is the process of deploying an application into production servers manually when it has completed testing and the build stages. Here, we'll automate the continuous integration processes, however, manual involvement is still required for deploying it to the production environment.

CONTAINERS

THAT ENSURE OUR APPLICATIONS RUN
CONSISTENTLY ACROSS ALL
ENVIRONMENTS.....!!

CONTAINERS

Why Use Containers in DevOps?

- Developers can run into some problems while moving the application from one place to another. For example, there may be problems like the software and configuration settings being different on the production end than the developer's end. These problems can occur any time there is a difference between the network, storage or security policies between the two environments.
- This is when DevOps Containers are used. As we already mentioned before, containers have all the necessary tools like binary codes, runtimes, and other files that are needed to run an application.
- You can run a Container in any environment as these things cannot be different in any environment. By their nature, they streamline the movement of the application from one environment to the other and make these problems disappear.



CONTAINERS

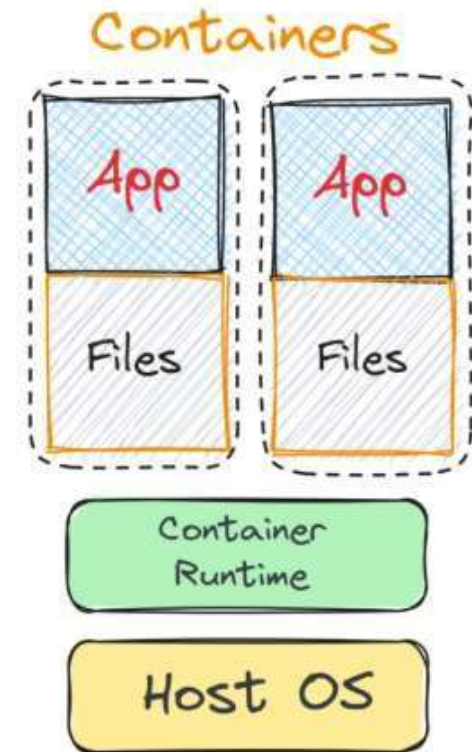
What Are Containers in DevOps?

- Containers are basically a type of OS virtualization. They provide a standard approach that you can use to package an application's important executables such as configuration files, libraries, binary codes, runtimes, and other system tools. However, unlike machine and server virtualization, containers do not carry the operating system images.
- This feature makes them very lightweight and they also have a lot fewer overheads.
- If there are multiple applications being deployed at once, then multiple containers can also be used in the form of a container cluster.

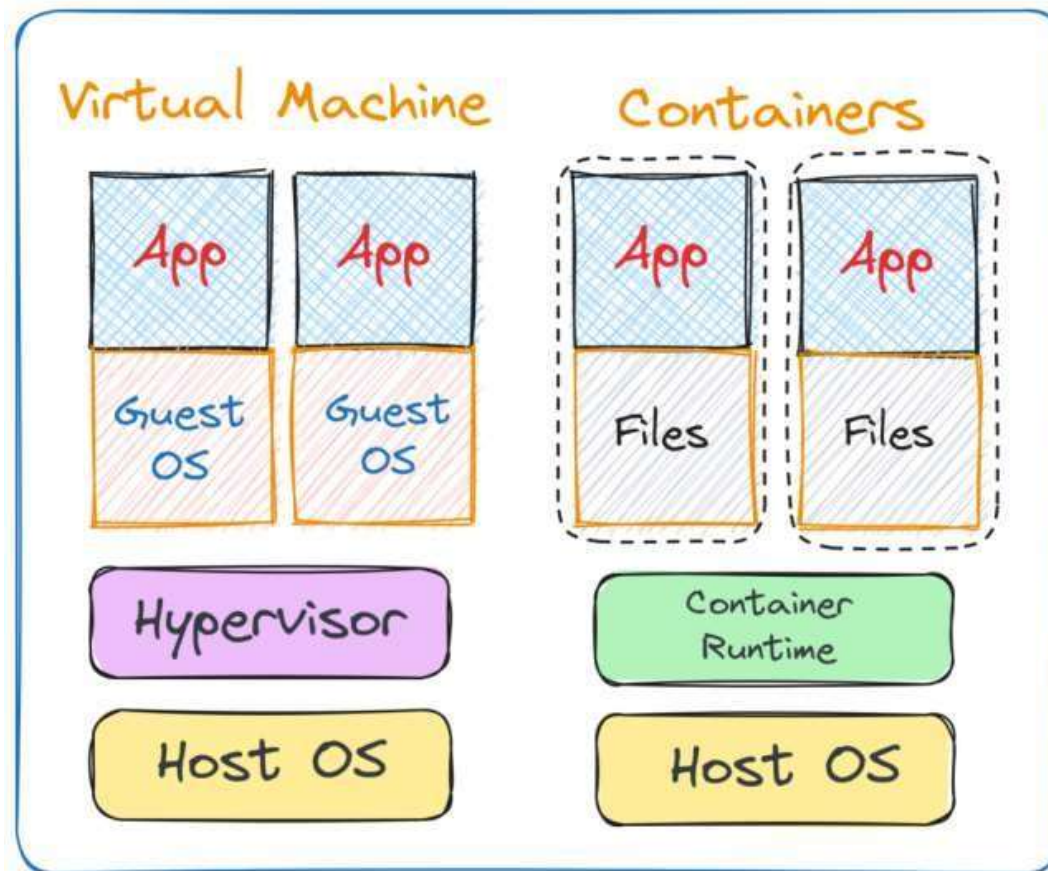


CONTAINER ARCHITECTURE

- Containers share the host operating system's kernel and run as isolated processes, utilizing lightweight operating system-level virtualization.
- They provide application-level isolation and resource allocation, allowing multiple containers to run on a single host.
- Containers have minimal overhead because they share the host's kernel and do not require the overhead of a separate guest operating system.
- This makes container startup and resource utilization faster and more efficient compared to VMs.



CONTAINER VS VIRTUAL MACHINES



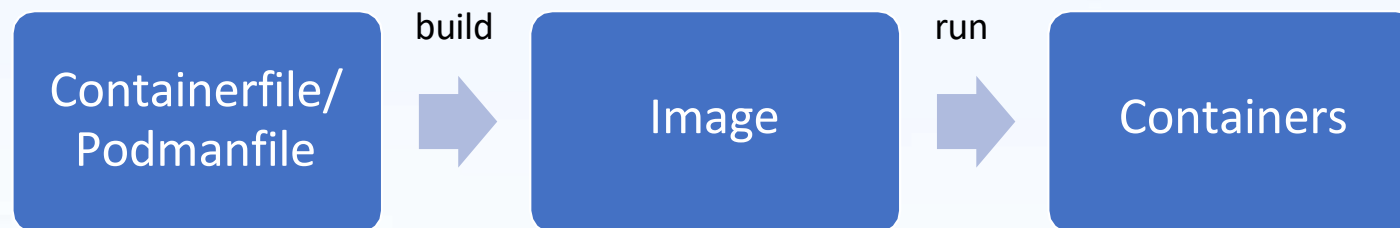
CONTAINER IMAGES-

CREATING CONTAINERS



WHAT IS A CONTAINER IMAGE?

- It is an executable package of software that includes everything needed to run an application.
- This image informs how a container should instantiate, determining which software components will run and how.
- Docker Container is a virtual environment that bundles application code with all the dependencies required to run the application.
- The application runs quickly and reliably from one computing environment to another.



CONTAINER VS IMAGE

CONTAINER	IMAGES
containers are a writable layer of Images.	Images consist of read-only layers.
containers are created using container Images. We can attach volumes, networks, etc. to the container at runtime.	Images are created by container file. We cannot attach volumes, networks, etc. to the Images as Images are immutable.
We cannot share containers to other team members directly, we need to first export it as an Image.	We can push the Docker images to the registry and share it with other team members.
In order to remove the container, container must be in running state.	In order to remove the image, the image must not be referenced by any container in any state.

STRUCTURE OF IMAGE

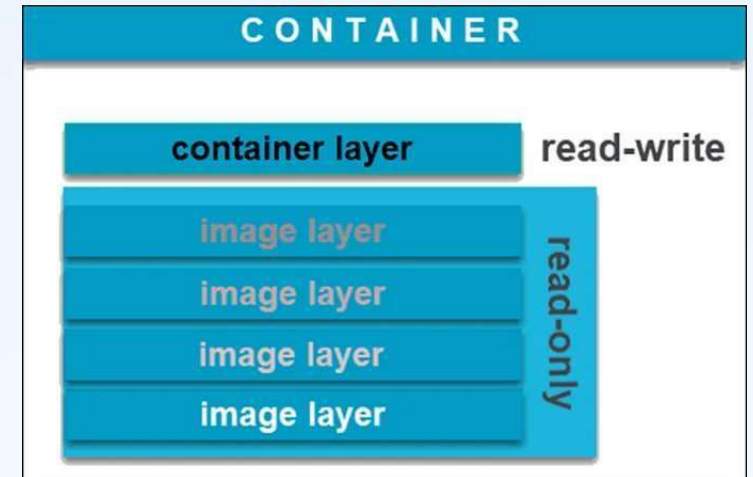
Base Image

Parent Image

Layers

Container layer

Container Registry





podman

THE CONTAINER RUN TIME

WHAT IS A PODMAN?

- **Podman** = Pod Manager → tool to create, build & run containers
- **Daemon-less** → no background service needed, more secure
- **Follows OCI standards** → works with standard container images
- **Production-ready** → manage containers easily in real environments
- **Lightweight & flexible** → suitable for developers and operations



HOW DOES PODMAN MANAGE CONTAINERS?

- **Daemonless Architecture**
 - No central daemon in background
 - Directly runs containers from CLI → reduces attack surface
- **Integration with system**
 - Containers managed like Linux services
 - Auto-start on boot, restart if failed
- **Rootless Operation**
 - Run containers without root privileges
 - Improves security and prevents full system compromise

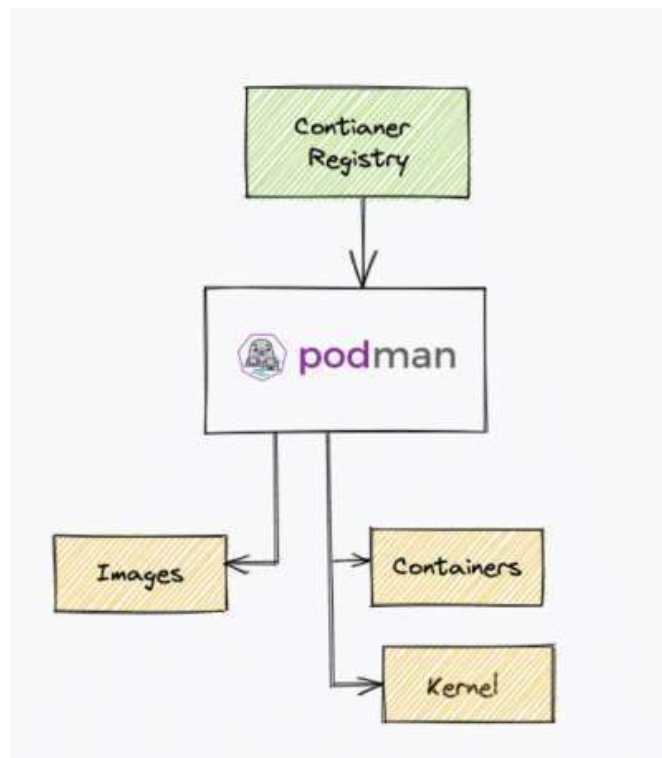
PLATFORMS FOR PODMAN

- Podman is primarily developed as an open-source project by the community, and it can be used on various Linux distributions.

1. Linux Distributions: Podman is designed to work natively on Linux. It is commonly used on popular Linux distributions such as:

1. Red Hat Enterprise Linux (RHEL)
2. Fedora
3. CentOS
4. Ubuntu
5. Debian
6. Many other Linux distributions

PODMAN ARCHITECTURE



QUIZ

1. What is Podman?

- A) An open-source container runtime tool
- B) A cloud computing platform
- C) A programming language for web development
- D) A version control system

QUIZ

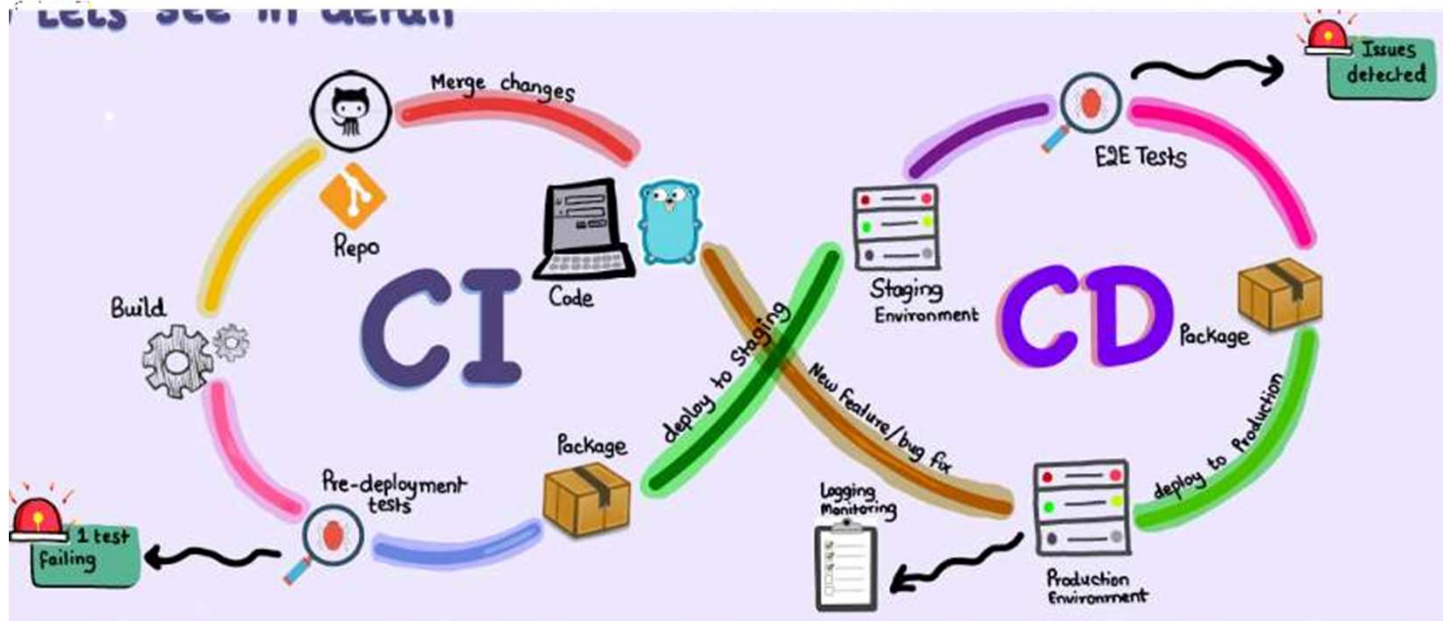
Question 2:

What is the purpose of a Pod in Podman?

- A) To manage and orchestrate containers
- B) To provide persistent storage for data
- C) To monitor container health and performance
- D) To manage network traffic between containers

Jenkins – Automating Workflows

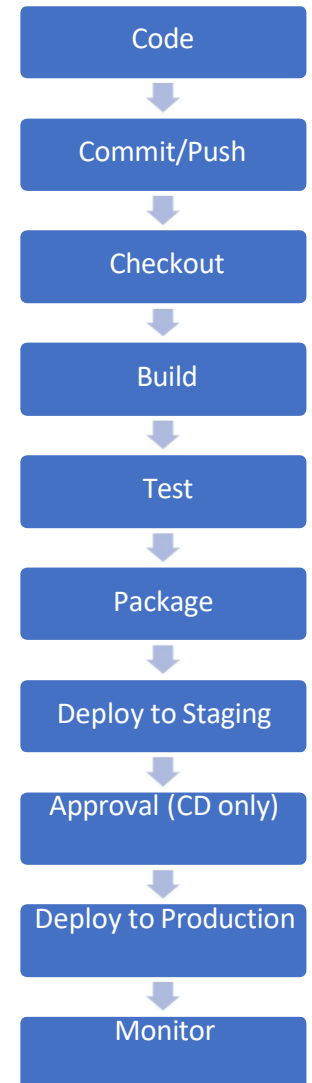
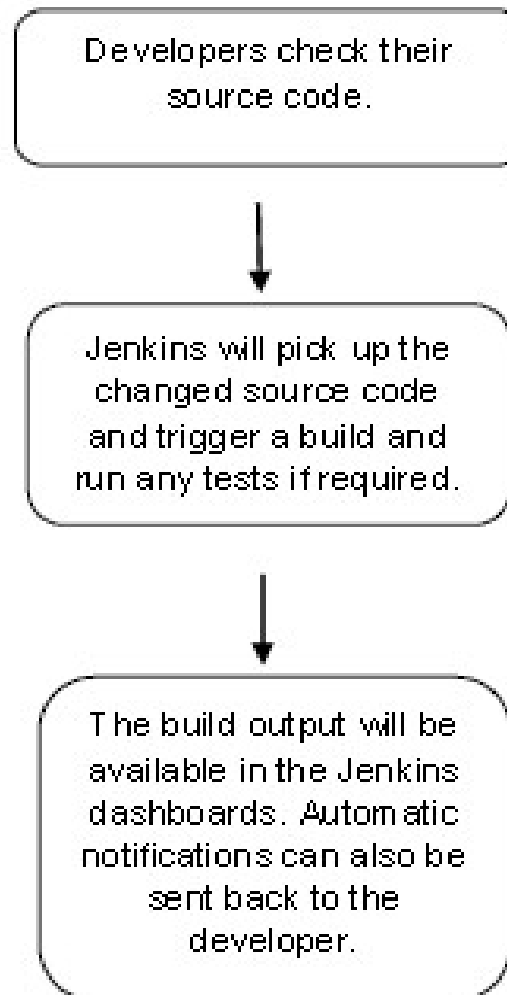
CI/CD PIPELINE



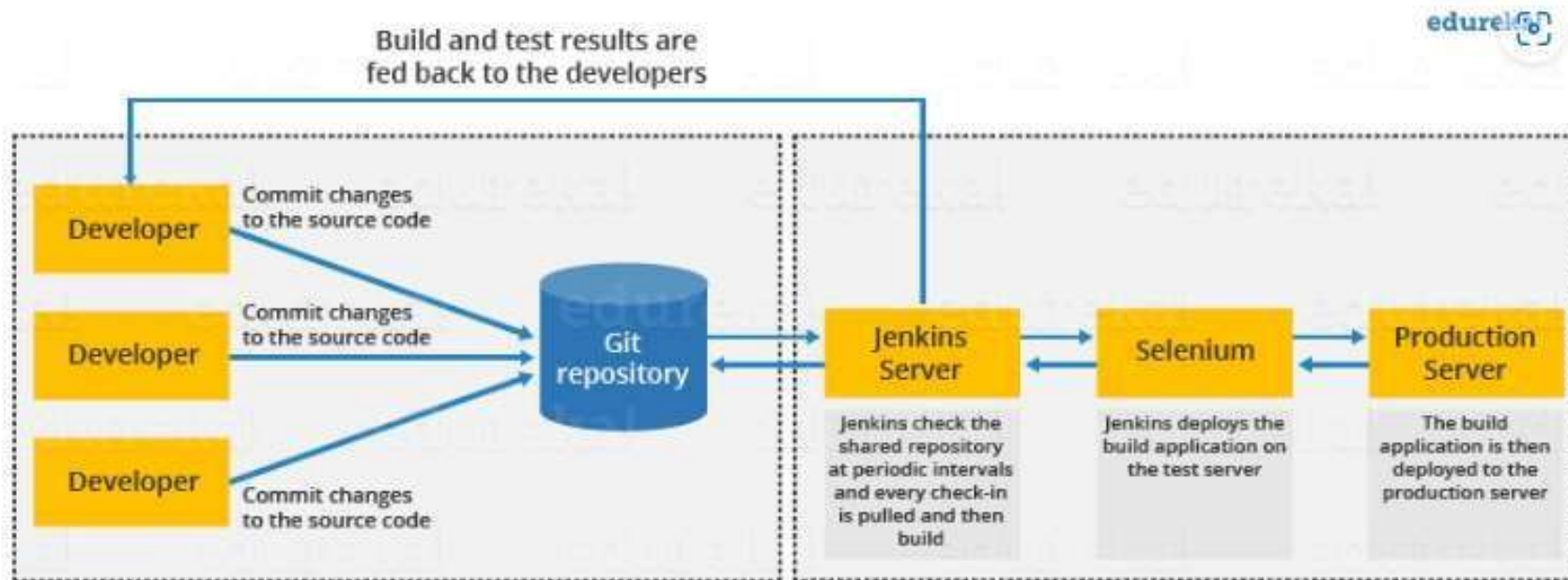
WHAT ARE JENKINS?

- Open-source automation tool.
- Automates **build, testing, and deployment**.
- Works with GitHub and other DevOps tools.
- Reduces manual effort and human errors.
- Helps deliver software faster.

So,
Jenkins automates repetitive software development tasks, making releases faster and more reliable.



CONTINUOUS INTEGRATION WITH JENKINS



JENKINS PIPELINE

- A Jenkins Pipeline is a **workflow** that automates the software delivery process.
- It divides the process into **multiple stages**, executed one after another.
- Each stage performs a **specific task**, making it easier to identify and fix errors.
- Pipelines are written as **code**, making them reusable and version-controlled.
- **Main Components:** Pipeline, Stage, Node.

LET US SEE AN EXAMPLE FOR MULTIPLE STAGES:

```
pipeline {  
  agent any  
  stages {  
    stage ('Build') {  
      ...  
    }  
    stage ('Deploy') {  
      ...  
    }  
    stage ('Monitor') {  
      ...  
    }  
  }  
}
```

Multiple Stages in Jenkins pipeline

Next Step: Integrating Testing into
Jenkins Pipelines ...!!!

TESTING

- Software testing is a process of identifying the correctness of software by considering its all attributes (Reliability, Scalability, Portability, Re-usability, and Usability) and evaluating the execution of software components to find software bugs or errors or defects.
- Testing is a group of techniques to determine the correctness of the application under the predefined script but, testing cannot find all the defects of the application. The main intent of testing is to detect failures in the application so that failures can be discovered and corrected.

TYPES OF TESTING

Manual Testing

Automate Testing

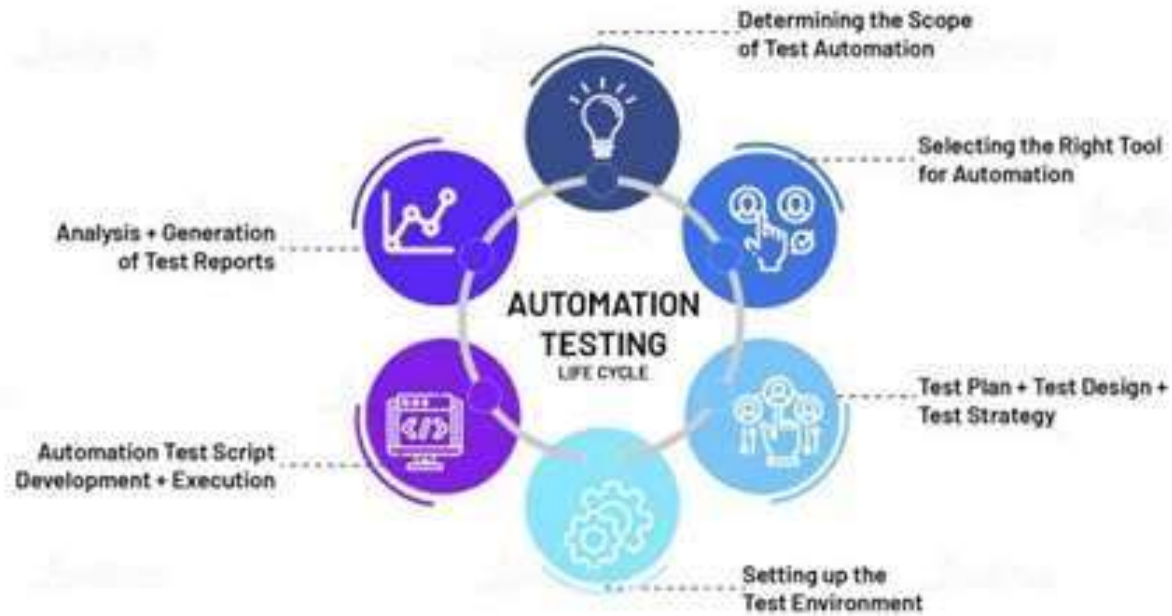
MANUAL TESTING

- Manual testing is a software testing process in which test cases are executed manually without using any automated tool.
- All test cases are executed by the tester manually according to the end user's perspective. It ensures whether the application is working, as mentioned in the requirement document or not.
- Test cases are planned and implemented to complete almost 100 per cent of the software application. Test case reports are also generated manually.

AUTOMATION TESTING

- **Automation Testing** is a software testing technique that performs using special automated testing software tools to execute a test case suite.
- On the contrary, Manual Testing is performed by a human sitting in front of a computer carefully executing the test steps.

AUTOMATION TESTING LIFE CYCLE



INTRODUCTION TO SELENIUM



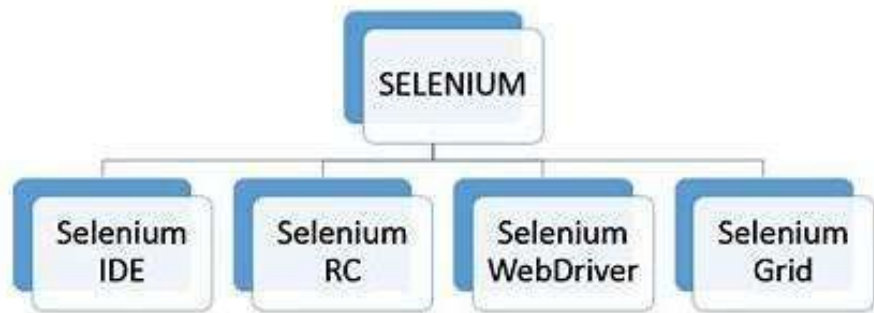
WHAT IS SELENIUM?

- **Open-source automation tool** for testing web applications.
- Simulates user actions like clicking buttons, entering data, and navigating web pages.
- Supports multiple browsers such as Chrome, Firefox, Edge, and Safari.
- Works on Windows, Linux, and macOS.
- Supports programming languages like Java, Python, C#, and JavaScript.

So,

Selenium automates web application testing, helping teams detect bugs early and improve software quality.

COMPONENTS



Selenium IDE – Records and plays back browser actions; ideal for beginners.

Selenium WebDriver – Automates web browsers using code; the most widely used Selenium component.

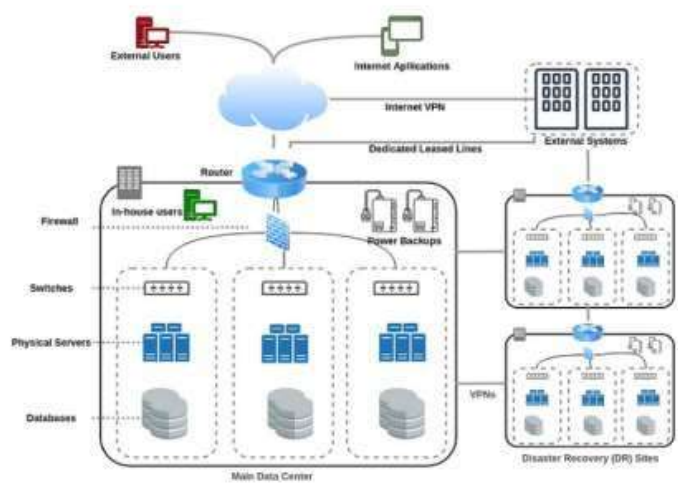
Selenium Grid – Runs tests on multiple browsers and machines simultaneously.

Selenium RC – Legacy component; replaced by Selenium WebDriver.

IAC AND TERRAFORM –

AUTOMATING INFRASTRUCTURE: IAC AND TERRAFORM

Pre-Cloud (1990s - 2000s)



Cloud (2010s - Now)



INFRASTRUCTURE AS A CODE

- Infrastructure As Code (IaC) is a methodology in which scripts automate infrastructure and configuration management.
- IaC tools allow you to abstract away details about your physical environment, allowing you to focus on what matters- your code.
- This approach is commonly used in cloud computing and data center environments to streamline and standardize infrastructure management.



WHAT PROBLEM DOES IaC SOLVES

- **Cost:** We have to hire many professionals to perform the necessary tasks at each step of the process, from network engineers to hardware maintenance technicians.
- **Scalability and Availability:** Everything falls down when it comes to Speed. Since manual configuration is so slow, applications would often struggle with spikes in access, while the system administrators would be desperately trying to set up servers to manage the load.
- **Inconsistency:** So, If you have several people manually deploying configurations, discrepancies aren't going to be unavoidable.



DIFFERENT APPROACHES TO INFRASTRUCTURE AS A CODE

- **Declarative approach** : defines the desired state of the system, including what resources you need and any properties they should have, and an IaC tool will configure it for you.
- **Imperative approach** : “gives orders.” It defines a sequence of commands or instructions so the infrastructure can reach the final result.

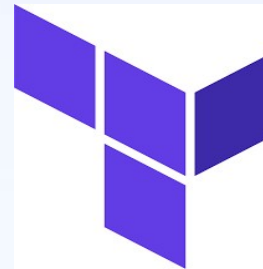


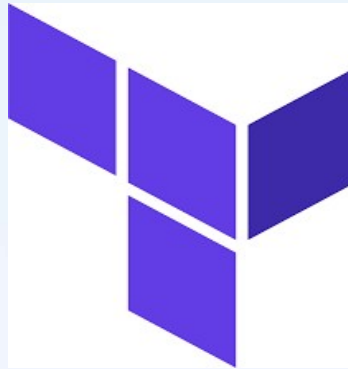
IAC PROVISIONING TOOLS

Cloud Specific



Cloud Agnostic





TERRAFORM

- Terraform is a tool for building, changing and versioning infrastructure safely and efficiently
- Compatible with clouds and services
- Mainly for infrastructure provisioning

WHAT IS TERRAFORM USED FOR?

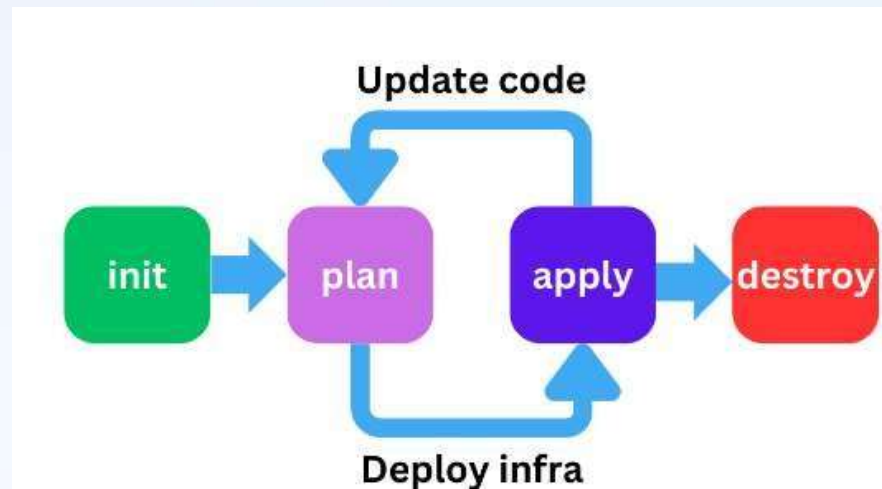
Public cloud provisioning

Multi cloud deployments

Deploying , managing and orchestrating resources



TERRAFORM LIFECYCLE



TERRAFORM LIFECYCLE (Contd.)

INIT COMMAND

- The working directory holding the Terraform configuration files is initialized using the Terraform init command.
- This is the first command that needs to be executed-

```
Command:  
# Initialize Terraform  
$ terraform init
```

PLAN COMMAND

- Reads the current state of any existing remote objects to ensure that the state is up to date.
- Compare the current state to the previous state, noting any differences.

```
Command:  
$ terraform plan
```



TERRAFORM LIFECYCLE (Contd.)

APPLY COMMAND

- Terraform apply executes the plan. This potentially changes the deployment.

```
Command:  
$ terraform apply
```

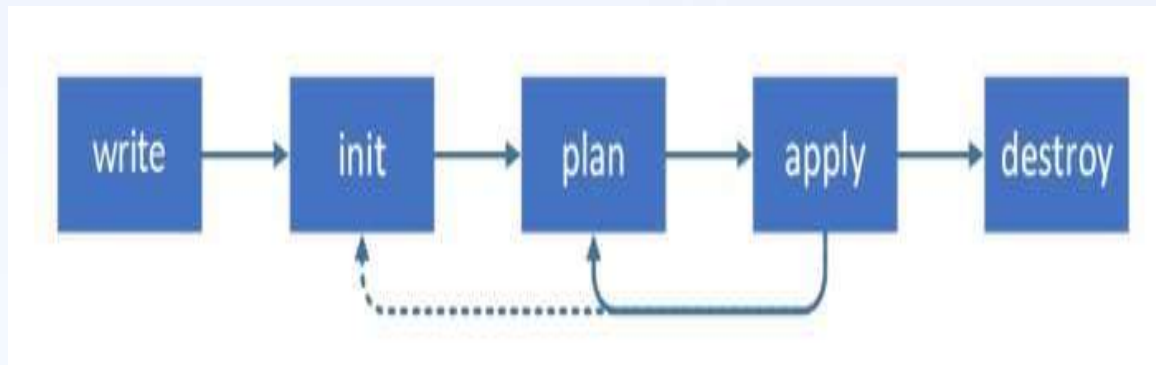
DESTROY COMMAND

- Terraform destroy deletes all resources that are governed by this specific terraform environment.

```
Command:  
$ terraform destroy
```



BASIC CONCEPT OF WORKFLOW



MONITORING



WHY MONITORING MATTERS ?

- Monitoring tells us **what's happening in real time** in our systems and applications.
- Helps detect **failures, performance issues, and anomalies** early.
- It **closes the feedback loop** in the DevOps lifecycle.



MONITORING VS OBSERVABILITY

Monitoring	Observability
Tracks known issues (alerts, logs)	Helps discover unknown issues
Answers: “ Is it working? ”	Answers: “ Why is it not working? ”
Uses metrics, logs, and traces	Uses data to understand internal state



TYPES OF MONITORING

- **Infrastructure Monitoring**
 - Monitors servers, CPUs, memory, networks (e.g., Terraform-provisioned resources)
- **Application Monitoring**
 - Tracks app health, performance (e.g., response time, error rates)
- **Log Monitoring**
 - Parses system logs for warnings, errors, and trends
- **User Monitoring**
 - Measures real user experience (e.g., page load time, click errors)



POPULAR MONITORING TOOLS

Tool	Use Case
Prometheus	Open-source metrics monitoring
Grafana	Visualizes data from Prometheus
ELK Stack	Log analysis (Elasticsearch, Logstash, Kibana)
Datadog	All-in-one cloud monitoring
New Relic	Application performance monitoring (APM)



MONITORING IN THE DEVOPS LIFECYCLE

- **How it fits:**
- After deployment (via Jenkins or Terraform), monitoring ensures:
 - System health
 - Fast feedback for incidents
 - Data for continuous improvement



SUMMARY

This one-day DevOps training gives a complete high-level view of how modern software development works using DevOps practices. You'll learn about:

- Automating builds and deployments using Jenkins
- Running apps in isolated environments using Docker
- Automating testing with Selenium
- Managing infrastructure with Terraform
- Monitoring systems for real-time feedback



Quiz

- **Q1. What is the main goal of DevOps?**
- A. Develop only the frontend application
- B. Improve collaboration between Development and Operations teams
- C. Replace developers with automation tools
- D. Manage databases only



- **Q2. In DevOps, the 'A' in CAMS stands for:**
- A. Analytics
- B. Assessment
- C. Automation
- D. Application

- **Q3. Which of the following is the correct order in a basic CI/CD pipeline?**
- A. Deploy → Build → Test → Checkout
- B. Checkout → Build → Test → Deploy
- C. Test → Deploy → Build → Checkout
- D. Build → Checkout → Deploy → Test

- **Q4. Which of the following is one of the 7 C's of DevOps?**
- A. Continuous Development
- B. Continuous Marketing
- C. Continuous Sales
- D. Continuous Networking

- **Q5. What is Continuous Integration (CI)?**
- A. Deploying applications automatically to production
- B. Regularly merging code changes into a shared repository and verifying the build
- C. Monitoring application performance after deployment
- D. Managing cloud infrastructure

- **Q6. Which Terraform command is used to preview infrastructure changes before applying them?**
- A. terraform deploy
- B. terraform execute
- C. terraform plan
- D. terraform destroy

- **Q7. Which type of testing verifies that individual components or functions work correctly?**
- A. System Testing
- B. Integration Testing
- C. Unit Testing
- D. Acceptance Testing

- **Q8. What is the primary purpose of Jenkins?**
- A. Create databases
- B. Automate build, testing, and deployment processes
- C. Design user interfaces
- D. Store application data

- **Q9. In a Jenkins Pipeline, which stage downloads the latest source code from GitHub?**
- A. Build
- B. Test
- C. Checkout
- D. Deploy

- **Q10. Which statement best describes Continuous Deployment?**
- A. Every successful change is automatically released to production without manual approval.
- B. Developers manually merge code every month.
- C. Applications are tested only after deployment.
- D. Infrastructure is created manually.